

Pourquoi je ne veux pas de Luciole

David LEGRAND

2026-07-29

Contexte

Ce document accompagne la restitution orale que je présente au groupe Mini Manifest, dans le cadre de la compétence C6 (veille) de ma certification. Le format prévu est de dix minutes de présentation suivies de vingt minutes de discussion ; ce texte en est la version écrite, destinée à être diffusée et relue plutôt que projetée telle quelle — le support de présentation (ODP) reste volontairement plus visuel et minimaliste.

Mon thème personnel pour ce Mini Manifest est le développement durable appliqué à l'intelligence artificielle, dans ses dimensions sociétale, environnementale et économique. Ce document raconte comment ce thème m'a mené, par un cheminement précis, à m'intéresser aux modèles de langage Lucie puis Luciole, et pourquoi je conclus, pour l'instant, à ne pas les adopter.

Mon thème, ma découverte

En creusant mon thème, je suis tombé sur la page officielle du collectif OpenLLM France. Ce qui m'a arrêté sur cette page, précisément dans l'axe de mon thème, c'est une analyse de cycle de vie revendiquée, de l'entraînement jusqu'à l'inférence, construite selon une méthodologie AF-NOR pour l'IA frugale. Ce n'est pas un projet de modèle de langage comme un autre : c'est un projet qui pose l'écologie comme critère de conception, et non comme argument marketing ajouté après coup. C'est cet angle frugalité et écologie — pas une curiosité générale pour les LLM déconnectée de mon thème assigné — qui m'a mené à leurs modèles : Lucie, puis Luciole.

OpenLLM France

OpenLLM France est une communauté et un consortium francophone créé à l'été 2023, mené par l'entreprise LINAGORA, dont l'objectif est de développer collaborativement un LLM français, souverain et réellement open source : données d'entraînement ouvertes, algorithmes documentés, licence non restrictive. Le consortium fondateur réunit GENCI, IDRIS (rattaché au CNRS), LORIA, CEA-List, l'université Paris 1 Panthéon-Sorbonne, le LIX de l'École Polytechnique, l'association Class'Code et la société Talkr.ai ; il s'est depuis élargi à plus de 300 organisations et plus de 800 membres individuels. Le projet est financé par BPI France dans le cadre de France 2030, aux côtés de la société Opsci.

LINAGORA n'est pas qu'un porteur ponctuel de projet IA : c'est un acteur établi du logiciel libre français, avec un catalogue de plus de 500 logiciels et des produits concrets déjà utilisés au quotidien — la suite collaborative Twake (messagerie, mail, stockage), LinShare pour le partage de fichiers, Linto pour la voix, LinID pour l'authentification. Son engagement sur les LLM souverains s'inscrit dans une mission ancienne d'indépendance technologique, pas dans un effet de mode.

Lucie, le lancement raté

Lucie est le premier modèle publié par le collectif, en janvier 2025, et son lancement a été chahuté. Cinq reproches lui ont été faits. D'abord des échecs sur des tests basiques : à la question du nombre de lettres dans un mot simple, ou sur des calculs élémentaires, le modèle s'est trompé. Ensuite l'absence d'alignement post-entraînement : sans RLHF (*Reinforcement Learning from Human Feedback*, un ajustement par apprentissage par renforcement à partir de retours humains, qui apprend au modèle à préférer les réponses jugées utiles plutôt que n'importe quelle suite de texte plausible) ni instruction tuning (un entraînement complémentaire sur des exemples de consignes et de réponses attendues, qui apprend au modèle à suivre une instruction plutôt qu'à simplement continuer un texte), Lucie savait produire du texte mais n'était pas encore un assistant utilisable en confiance. Troisième reproche, une infrastructure sous-dimensionnée : l'interface publique lucie.chat a connu jusqu'à 45 minutes d'attente, la charge du lancement n'ayant pas été anticipée. Quatrième point, une communication en décalage avec la réalité du projet : une annonce portée par Éduscol présentait Lucie avec un optimisme qui ne mentionnait pas qu'il s'agissait d'un projet de recherche académique en phase initiale. Enfin, le timing même du lancement, un vendredi, sans équipe mobilisée pour gérer les retours du week-end, alors que la viralité négative a été immédiate.

Luciole, la réponse et le tilt RAG

Luciole, publié en juin 2026, répond explicitement à chacun de ces reproches. Il est repositionné comme des briques brutes destinées à l'adaptation métier, pas comme un produit grand public — le président de LINAGORA le dit lui-même : une approche scientifique, pas commerciale. Trois tailles sont proposées selon les cas d'usage : 1B pour l'edge, 8B pour un contexte long, 23B pour le raisonnement, plutôt qu'un modèle unique cherchant à tout faire. La transparence est totale : les poids, les scripts d'entraînement et le corpus sont publiés séparément, sous trois licences distinctes, ce qui rend le modèle reproductible et évaluable indépendamment. Et l'alignement, absent chez Lucie, est cette fois documenté : la variante Instruct a reçu un entraînement en trois phases, du SFT avec et sans traces de raisonnement, puis du DPO.

Le point qui m'intéresse le plus dans cette veille est ailleurs : la fiche technique du modèle Luciole-23B-Instruct documente explicitement le RAG comme cas d'usage, et recommande d'augmenter le modèle par de la récupération documentaire plutôt que de compter sur ses connaissances internes. Or je dispose déjà d'un outil de recherche sémantique sur un domaine que je maîtrise personnellement : `dirty_retriever`, qui interroge les 245 règles du référentiel Opquast utilisé par QualiCheck. C'était l'occasion d'un terrain de test concret, pas un exemple jouet. Je tiens à préciser que cet essai reste une démonstration pour le groupe Mini Manifest, pas une décision d'architecture pour QualiCheck : aucun changement n'est envisagé sur le pipeline de production.

Setup du test

Pour mener cet essai, j'ai déployé Ollama dans un conteneur Docker sur un serveur personnel, cloclo, qui dispose de 32 Go de RAM. Mon PC de travail, avec seulement 5,9 Go de RAM réellement disponibles sur ses 15,8 Go, était trop juste pour charger un modèle de 8 milliards de paramètres dans de bonnes conditions. La configuration Docker reste minimale : l'image officielle ollama/ollama, un volume persistant pour les modèles téléchargés, et le port 11434 exposé pour les requêtes.

Latence : premiers résultats

Le premier test, volontairement trivial — une simple question « Salut » — visait à isoler le coût de base d'un appel, indépendamment de la charge d'un contexte RAG. Le premier appel à Luciole 8B a mis 15,57 secondes à répondre. Le deuxième appel, sur un modèle cette fois déjà chargé en mémoire, a mis 17,86 secondes : encore plus lent que le premier, ce qui élimine l'hypothèse d'un simple chargement à froid. En comparaison, Mistral 7B, déjà chaud au moment du test, a répondu en 6,39 secondes sur un premier essai rapide, et en 9,26 secondes sur une réponse plus longue et mieux mesurée. L'écart est réel, pas un artefact de mesure.

Vérification CPU

Avant de conclure quoi que ce soit sur l'architecture du modèle, j'ai voulu écarter une explication plus triviale : un problème de ressources CPU allouées. La commande **docker exec ollama nproc** confirme que huit cœurs sont disponibles côté conteneur. Pendant les appels, **docker stats** montre un taux d'utilisation qui plafonne à 400 %, soit quatre cœurs sur huit, et ce de façon identique pour les deux modèles. L'allocation de ressources est donc strictement la même dans les deux cas : ce n'est pas là que se situe l'écart.

Le chiffre qui compte vraiment est le temps de **prompt_eval**, c'est-à-dire le prefill : le traitement initial du prompt avant toute génération de réponse. Sur ce point précis, Luciole traite son prompt à 134 millisecondes par token, contre 32 millisecondes par token pour Mistral. Le goulot d'étranglement n'est donc pas la génération de la réponse, mais bien ce traitement initial du prompt.

Transformer : l'attention et son coût quadratique

Pour comprendre cet écart, il faut revenir à l'architecture des modèles. Un Transformer classique — celui de Mistral — traite un prompt grâce à un mécanisme d'attention : chaque token du texte regarde directement tous les autres tokens du même prompt, en une seule passe, plutôt que de les lire séquentiellement. Cette mise en regard exhaustive de tous les tokens entre eux a toutefois un coût qui grandit au carré de la longueur du contexte : doubler la taille du prompt quadruple le coût de calcul de l'attention.

Mamba / SSM : le pari du coût linéaire

Face à ce coût quadratique, une autre famille de modèles propose un compromis différent : les State Space Models, ou SSM, un formalisme emprunté à l'automatique et au traitement du signal. Mamba, publié par Albert Gu et Tri Dao en décembre 2023, en est la variante la plus connue en traitement du langage. L'idée centrale est de ne pas garder un accès direct à chaque token passé comme le fait l'attention, mais de résumer tout l'historique dans un état interne de taille fixe, mis à jour token par token — une logique plus proche, sur ce point précis, d'un réseau de neurones récurrent classique que d'un Transformer.

La conséquence recherchée est un coût qui grandit linéairement avec la longueur du contexte, et non au carré : un contexte deux fois plus long devrait coûter environ deux fois plus cher à traiter, pas quatre fois. Luciole 8B est précisément un modèle hybride, mêlant des couches Transformer et des couches Mamba à la manière du modèle Jamba, dans l'idée de conserver la qualité de raisonnement du Transformer tout en profitant de l'efficacité de Mamba sur les contextes longs.

CPU vs GPU

Un GPU dispose de milliers de petits cœurs, taillés pour effectuer le même calcul en parallèle sur une grande quantité de données indépendantes — exactement la forme du calcul d’attention d’un Transformer. Un CPU, à l’inverse, dispose de peu de cœurs, puissants mais conçus pour un traitement séquentiel, pas à pas. Cela ne signifie pas pour autant qu’un CPU soit intrinsèquement inadapté à l’intelligence artificielle en général : CroissantLLM, un modèle de 1,3 milliard de paramètres développé par le laboratoire MICS de CentraleSupélec avec Illuin Technology, est un Transformer pur présenté par ses auteurs comme capable de tourner sur CPU, et même sur téléphone, précisément parce qu’il reste très compact.

Mamba, le constat

Le constat, chez moi, en pratique, est que ce gain théorique promis par Mamba ne s'est pas vérifié sur ce test précis : le prefill de Luciole est resté aussi lent, par token, que sa propre génération, alors que Mistral traite son prompt environ six fois plus vite qu'il ne génère sa réponse. Sur ce test, sur CPU, l'efficacité annoncée de Mamba ne s'est donc pas traduite dans les chiffres.

Une réserve s'impose néanmoins : ce résultat est probablement spécifique à cette inférence CPU précise. L'implémentation de référence de Mamba est pensée pour tourner sur GPU, avec un algorithme de parallélisation dit *parallel scan*, conçu pour la hiérarchie mémoire de ce matériel ; si ce mécanisme n'est pas implémenté, ou mal implémenté, dans le moteur d'inférence utilisé ici (GGUF via llama.cpp, sur CPU), le prefill retombe à un calcul quasi séquentiel — ce qui correspond à ce que j'ai observé empiriquement. Le détail complet de cette hypothèse figure dans la fiche consacrée à l'architecture Mamba, disponible parmi les fiches de recherche qui accompagnent ce document.

Conclusion

Je ne retiens donc pas Luciole pour l'instant : sur ce test précis, l'efficacité théorique de l'architecture Mamba ne s'est pas vérifiée en pratique sur une inférence CPU. Apertus-70B, autre candidat souverain et déjà retenu en production pour le RAG de QualiCheck via l'API Infomaniak, est lui aussi écarté de ce comparatif auto-hébergé, mais pour une tout autre raison : même quantifié, un modèle de 70 milliards de paramètres ne tient pas raisonnablement dans les 32 Go de RAM de mon serveur personnel, et serait de toute façon surdimensionné pour la simple synthèse de trois règles déjà retrouvées par la recherche documentaire.

La piste que je retiens pour la suite est CroissantLLM : un modèle de 1,3 milliard de paramètres, bilingue français-anglais dès sa conception, et Transformer pur — donc sans le facteur confondant que représente l'architecture Mamba de Luciole. Un comparatif sur un terrain plus équitable reste à mener, en gardant à l'esprit que sa petite taille lui donne un avantage de latence presque par construction face à Mistral 7B ou à Luciole 8B.